

0.18. Avancer avec `deplace_x`

`deplace_x(colonne, ligne, tuile, pas)` fait tout le 0.17 en une ligne : la lettre avance de 5 cases.

1. Ce qu'il faut comprendre

C'est le programme du 0.17, mais tout le travail (le chronomètre, l'effacement, le pas de plus, la lettre reposée) est fait par **une seule fonction de la console** : `deplace_x`.

Ce qui est nouveau ici : `deplace_x(colonne, ligne, tuile, pas)`. Elle pose la tuile en (`colonne`, `ligne`), puis la fait avancer d'une case vers la droite tous les quarts de seconde (15 images), **pas** fois.

Pourquoi `x = deplace_x(...)` ? Quand on écrit `deplace_x(x, ...)`, la fonction ne reçoit pas la variable `x` elle-même, mais une **copie** de sa valeur (0). Elle fait bouger la lettre en changeant sa copie, jamais notre `x`. À la fin, elle **rend** (`return`) la colonne d'arrivée, 5, et `x = ...` range ce 5 dans `x`.

Sans le `x =`, en C ordinaire, la lettre bougerait bien à l'écran, mais `x` vaudrait encore 0 : le programme croirait la lettre toujours au départ. Le cours suivant montre quand cela pose problème.

Elle bloque, comme `attendre()` : pendant le voyage, rien d'autre ne tourne. Au bord de l'écran (colonne 19), elle s'arrête au lieu de sortir.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1595 octets** de cartouche.

1. UNE VARIABLE — PAS

```
uint8_t pas = 5;    // le nombre de cases à parcourir
```

2. UNE VARIABLE — X

```
uint8_t x = 0;      // la colonne de la lettre : elle part de 0 (tout à gauche)
```

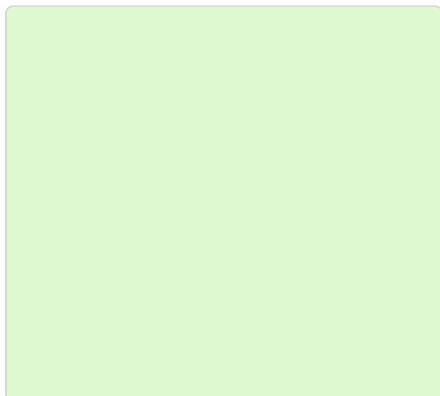
3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // La ligne, morceau par morceau :
    //
    // x = deplace_x(x, 0, ALPHABET[0], pas);
    // | | | | |
    // | | | | | +-- pas : combien de cases (5)
    // | | | | | +----- tuile : ce qui bouge, la lettre A
    // | | | | | +----- ligne : 0, tout en haut
    // | | | | | +----- colonne : d'où la lettre part.
    // | | | | | La fonction reçoit une COPIE
    // | | | | | de x (sa valeur, 0), pas x.
    // | +--- deplace_x fait avancer la lettre, puis REND (return) la
    // | colonne d'arrivée : 5.
    // +----- « x = » range ce 5 dans x.
    //
    // POURQUOI « x = » ? Une fonction n'a qu'une COPIE de x : elle ne peut pas
    // le changer. Pour garder le résultat, on le RANGE nous-mêmes : x = ...
    x = deplace_x(x, 0, ALPHABET[0], pas); // après cette ligne : x vaut 5

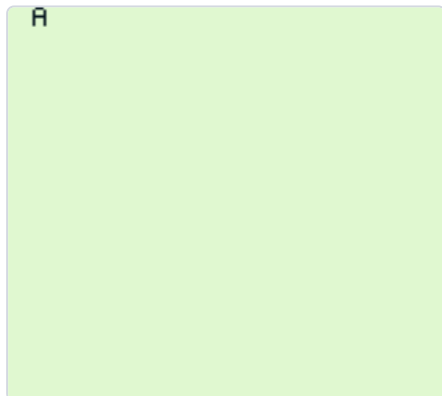
    while (true) {
        image(); // le programme continue ; la lettre, elle, reste en colonne 5
    }
}
```

3. Ce qui se passe, étape par étape

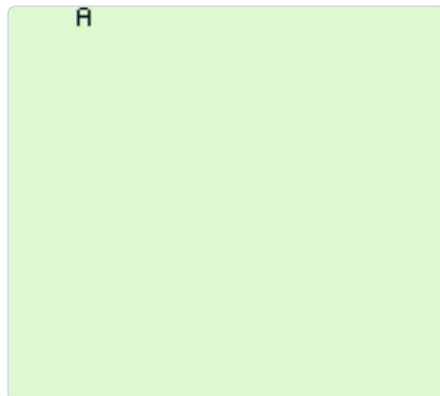
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un A qui avance de 5 cases vers la droite, puis s'arrête en colonne 5.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ x a reçu la colonne d'arrivée ($x = 5$)
- ✓ le A est en colonne 5, seul sur sa ligne (« A »)

Ensuite : **0.18.1. Avancer avec `deplace_x` — de base, ailleurs** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++